

Exercise 5

1. Write a simple program where the source and the destination doesn't match. You can take either message1.cc or message2.cc and modify the program. What happens in this situation?

```
[ec2-user@ip-172-31-20-184 lab8]$ mpicxx -o mismatch mismatch.cc
mpirun --timeout 10 --oversubscribe -np 3 ./mismatch; echo "exit status = $"
mpirun --oversubscribe -np 2 ./mismatch
Process 1 waiting for a message from process 2
Process 0 sent 42 to process 1
-----
The user-provided time limit for job execution has been reached:

Timeout: 10 seconds

The job will now be aborted. Please check your code and/or
adjust/remove the job execution time limit (as specified by --timeout
command line option or MPIEXEC_TIMEOUT environment variable).
-----
exit status = 110
Process 0 sent 42 to process 1
Process 1 waiting for a message from process 2
[ip-172-31-20-184:36903] *** An error occurred in MPI_Recv
[ip-172-31-20-184:36903] *** reported by process [3239575553,1]
[ip-172-31-20-184:36903] *** on communicator MPI_COMM_WORLD
[ip-172-31-20-184:36903] *** MPI_ERR_RANK: invalid rank
[ip-172-31-20-184:36903] *** MPI_ERRORS_ARE_FATAL (processes in this communicator will now abort,
[ip-172-31-20-184:36903] *** and potentially your MPI job)
[ec2-user@ip-172-31-20-184 lab8]$
```

message1.cc was changed (mismatch.cc) so that rank 0 still sends the number to rank 1, but rank 1 calls MPI_Recv with source = 2 instead of 0. With 3 processes MPI_Send on rank 0 returns (the message is small, so it is buffered) and "Process 0 sent 42 to process 1" is printed, but rank 1 is waiting for a message from rank 2 and rank 2 never sends one. MPI_Recv is a blocking call and the message from rank 0 does not match the requested source, so rank 1 blocks forever. There is no error message; the program just hangs and never finishes. It was stopped after 10 seconds with the timeout option of mpirun (exit status = 110).

When the same program is run with only 2 processes, rank 2 does not exist, so MPI_Recv fails immediately with MPI_ERR_RANK: invalid rank and the job is aborted. With MPI_Ssend instead of MPI_Send the sender blocks as well, because a synchronous send only returns after the matching receive has started.

2. Rewrite message2.cc using Buffered Send (BSend) ensure that you do not replace the variables that you are using to send the data.

```
[ec2-user@ip-172-31-20-184 lab8]$ mpicxx -o message2 message2.cc
mpirun --oversubscribe -np 2 ./message2
mpicxx -o message2_bsend message2_bsend.cc
mpirun --oversubscribe -np 2 ./message2_bsend
Process 0 sent 0
Process 0 sent 10
Process 0 sent 20
Process 1 received 0
Process 1 received 10
Process 1 received 20
Process 0 sent 0
Process 0 sent 10
Process 0 sent 20
Process 1 received 0
Process 1 received 10
Process 1 received 20
[ec2-user@ip-172-31-20-184 lab8]$ |
```

MPI_Send was replaced with MPI_Bsend (message2_bsend.cc). Before the loop rank 0 attaches its own buffer with MPI_Buffer_attach. The buffer is sized for all three messages, $3 \times (\text{MPI_Pack_size of one MPI_INT} + \text{MPI_BSEND_OVERHEAD})$, so a later message never overwrites one that is still waiting in the buffer. The first six lines of the output are from the original message2.cc and the last six are from the buffered version.

The same variable number is still used for all three sends. MPI_Bsend copies the value into the attached buffer before it returns, so number can be changed in the next iteration without affecting the messages that were already sent. To show this, rank 1 sleeps for 1 second before it starts receiving: all three sends finish first and rank 1 still receives 0, 10 and 20. MPI_Buffer_detach is called after the loop, which waits until all the buffered messages have been delivered before the buffer is freed.